Final Project: Distributed Semantic Retrieval System

SENG 468: Software System Scalability
Spring 2026

Instructor: Dr. Ardeshir Shojaeinasab

Released: February 10, 2026 (Week 6) — Checkpoint: March 25, 2026 — Due: April 20, 2026

Project Overview: Build a scalable semantic search engine that handles concurrent users uploading PDFs and searching for relevant content. This is your capstone demonstration of everything you've learned about web scalability, asynchronous processing, distributed systems, and performance optimization.

Team Size: 2 students (strict)
Total Points: 100 points + up to 20 bonus points

Contents

1 What You're Building **3**
1.1 What This Is NOT . . . 3
1.2 Example User Flow . . . 3

2 Learning Objectives **3**

3 Technical Requirements **4**
3.1 Core Functionality Requirements . . . 4
3.2 Technology Guidance (Not Requirements) . . . 5

4 API Specification (Port 8080) **5**
4.1 Authentication Endpoints . . . 6
4.2 Document Management Endpoints . . . 6
4.3 Search Endpoint . . . 8

5 System Architecture Guidelines **9**
5.1 Required Architecture Patterns . . . 9
5.2 Example Architecture Diagram . . . 9

6 Performance Evaluation **10**
6.1 Load Testing Requirements . . . 10
6.2 What You Must Report . . . 11

7 Deliverables **11**
7.1 Docker Compose (Mandatory) . . . 11
7.2 Technical Report (Mandatory Prerequisite) . . . 12
7.3 Presentation Video (15%) . . . 13
7.4 GitHub Repository (15%) . . . 14

8 Grading Rubric **15**
8.1 Design & Implementation (40 points) . . . 15
8.2 Performance & Load Testing (15 points) . . . 16
8.3 GitHub & Collaboration (15 points) . . . 16
8.4 Presentation Video (15 points) . . . 17
8.5 Checkpoint: March 25, 2026 (15 points) . . . 17
8.6 Bonus Points (Up to 20 points) . . . 18

9 Timeline **18**

10 GitHub Requirements **19**
10.1 Repository Setup . . . 19
10.2 Required Files . . . 19
10.3 Branch Strategy . . . 20
10.4 Commit Best Practices . . . 20
10.5 What NOT to Commit . . . 20

11 Report Structure Guidance **21**
11.1 Required Report Sections . . . 21

12 AI Usage Policy **22**
12.1 AI Tools Are Allowed With Disclosure . . . 22
12.2 Core Principle: You Control AI, Not Vice Versa . . . 22
12.3 Mandatory Disclosure . . . 23
12.4 Understanding Is Required . . . 23
12.5 Acceptable vs. Unacceptable AI Use . . . 23

13 Tips for Success **24**
13.1 Start Early . . . 24
13.2 Incremental Development . . . 24
13.3 Test Locally First . . . 25
13.4 Communication . . . 25
13.5 Common Pitfalls to Avoid . . . 25

14 Frequently Asked Questions **25**

15 Submission **26**
15.1 What to Submit (April 20, 2026) . . . 26
15.2 GitHub Repository Must Contain . . . 26
15.3 Late Policy . . . 26

16 Academic Integrity **27**

1 What You're Building

You will create a **distributed semantic search system** where:

1. Users can **sign up and log in** (authentication required)
2. Users can **upload PDF documents** (personal document library)
3. Users can **search across their PDFs** using natural language queries
4. System returns the **top 5 most relevant paragraphs** with relevance scores
5. System handles **concurrent users** uploading and searching simultaneously
6. Everything runs via **Docker Compose** (single command deployment)

1.1 What This Is NOT

- **NOT a RAG system** (no LLM-generated answers, just retrieval)
- **NOT a keyword search** (use semantic embeddings, not regex/SQL LIKE)
- **NOT about UI beauty** (simple functional UI is fine, focus on backend)

1.2 Example User Flow

1. Alice signs up and logs in
2. Alice uploads 3 research papers (PDFs)
3. Alice searches: *"machine learning optimization techniques"*
4. System returns 5 paragraphs sorted by relevance score:
   - Paragraph from Paper 1 (score: 0.89)
   - Paragraph from Paper 3 (score: 0.82)
   - Paragraph from Paper 2 (score: 0.78)
   - ...

2 Learning Objectives

By completing this project, you will demonstrate mastery of scalability concepts. Depending on your design choices, you **may** demonstrate:

- **Asynchronous processing** (decoupling heavy tasks from API responses)
- **Message queues** (RabbitMQ, Redis Queue, or equivalent)
- **Caching strategies** (Redis or equivalent for performance optimization)
- **Object storage** (MinIO for stateless file handling)
- **Vector databases** (ChromaDB, Qdrant, Milvus, pgvector, Elasticsearch, FAISS, OpenSearch)

- **Semantic embeddings** (sentence-transformers or equivalent local models)
- **Horizontal scalability** (stateless design, load balancing, database sharding)
- **Database optimization** (indexing, query optimization, connection pooling)
- **Performance testing** (Locust, k6, or equivalent)
- **Docker Compose orchestration** (multi-container deployment)
- **Professional Git workflow** (feature branches, pull requests, code reviews)

Note: The technologies and patterns you implement depend on your architectural decisions. What matters is that your system scales horizontally and handles concurrent load effectively.

3 Technical Requirements

3.1 Core Functionality Requirements

Your system MUST implement the following:

1. **User Authentication**
   - Sign up (username + password)
   - Login (returns token: JWT or session ID)
   - All document operations require authentication
2. **PDF Management**
   - Upload PDFs (multipart/form-data)
   - List user's uploaded PDFs
   - Delete PDFs (removes file and all vector data)
   - PDFs are user-specific (Alice cannot see Bob's PDFs)
3. **Semantic Search**
   - Search across user's PDFs using natural language
   - Return top 5 most relevant paragraphs
   - Each result includes: paragraph text, relevance score, source document ID
   - Results sorted by relevance (highest first)
4. **Asynchronous Processing**
   - PDF upload returns immediately (does not wait for processing)
   - PDF parsing and embedding generation happen in background
   - Use message queue or task queue pattern

3.2 Technology Guidance (Not Requirements)

You have **full autonomy** in technology choices. However, here are suggestions based on scalability best practices:

Suggested Technologies & Why:

- **MinIO for PDF Storage**
  - **Note:** I recommend further research about object storages like MinIO as this is not covered in lectures
  - S3-compatible object storage
  - Keeps your API tier stateless (no local file storage)
  - Enables horizontal scaling (multiple API instances can access same storage)
  - Industry standard for microservices architecture
- **Vector Database for Embeddings**
  - Options: ChromaDB, Qdrant, Milvus, pgvector, Elasticsearch, FAISS, OpenSearch
  - Optimized for similarity search (cosine similarity, dot product)
  - Much faster than computing distances for every paragraph
  - Essential for semantic search at scale
- **Message Queue for Async Processing**
  - RabbitMQ (covered in Lab 7) or Redis Queue
  - Decouples API from CPU-intensive PDF processing
  - Allows multiple worker processes
  - Prevents API timeouts on large PDFs
- **Embedding Models (Local Only)**
  - sentence-transformers (e.g., `all-MiniLM-L6-v2`)
  - Any local transformer library that produces vector embeddings
  - **Do not use API-based models** (e.g., OpenAI) - we need to stress test without cost concerns
  - Model must run locally in your Docker environment

Key Point: Whatever technology stack you choose, if it is backed by reasoning and logic and works at scale horizontally, it works for us. **Justify your choices in your report.**

4 API Specification (Port 8080)

CRITICAL: All systems **MUST** expose these exact endpoints on **Port 8080**. TAs will use automated scripts to test your API. Non-compliance will result in test failures and point deductions.

4.1 Authentication Endpoints

POST /auth/signup

Create a new user account

Request:

```
{
  "username": "alice",
  "password": "securepassword123"
}
```

Response (200 OK):

```
{
  "message": "User created successfully",
  "user_id": "uuid-or-integer"
}
```

Error (409 Conflict):

```
{
  "error": "Username already exists"
}
```

POST /auth/login

Authenticate and receive token

Request:

```
{
  "username": "alice",
  "password": "securepassword123"
}
```

Response (200 OK):

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user_id": "uuid-or-integer"
}
```

Error (401 Unauthorized):

```
{
  "error": "Invalid credentials"
}
```

Note: Use this token in `Authorization: Bearer <token>` header for all subsequent requests.

4.2 Document Management Endpoints

POST /documents

Upload a PDF (requires authentication)

Request:

- Content-Type: `multipart/form-data`
- Form field name: `file`
- Header: `Authorization: Bearer <token>`

Response (202 Accepted):

```
{
  "message": "PDF uploaded, processing started",
  "document_id": "uuid-123",
  "status": "processing"
}
```

Note: Return 202 (Accepted) immediately. Do NOT wait for processing to complete.

GET /documents
List all user's documents (requires authentication)
Response (200 OK):

```
[
  {
    "document_id": "uuid-123",
    "filename": "research_paper.pdf",
    "upload_date": "2026-03-15T10:30:00Z",
    "status": "ready",
    "page_count": 12
  },
  {
    "document_id": "uuid-456",
    "filename": "textbook_chapter.pdf",
    "upload_date": "2026-03-16T14:22:00Z",
    "status": "processing",
    "page_count": null
  }
]
```

DELETE /documents/{id}
Delete a document and all associated data (requires authentication)
Response (200 OK):

```
{
  "message": "Document and all associated data deleted",
  "document_id": "uuid-123"
}
```

Error (404 Not Found):

```
{
  "error": "Document not found or not owned by user"
}
```

4.3 Search Endpoint

GET /search?q={query}

Search across user's documents (requires authentication)

Request:

- Query parameter: `q` (URL-encoded search query)
- Header: `Authorization: Bearer <token>`

Example: `GET /search?q=machine%20learning%20optimization`

Response (200 OK):

```
[
  {
    "text": "Machine learning optimization techniques include gradient descent, Adam optimizer, and stochastic methods that improve convergence rates significantly.",
    "score": 0.942,
    "document_id": "uuid-123",
    "filename": "research_paper.pdf"
  },
  {
    "text": "Optimization in neural networks requires careful tuning of hyperparameters such as learning rate and batch size.",
    "score": 0.867,
    "document_id": "uuid-456",
    "filename": "textbook_chapter.pdf"
  },
  {
    "text": "Another relevant paragraph about optimization...",
    "score": 0.812,
    "document_id": "uuid-123",
    "filename": "research_paper.pdf"
  },
  {
    "text": "Fourth relevant paragraph...",
    "score": 0.755,
    "document_id": "uuid-789",
    "filename": "ml_handbook.pdf"
  },
  {
    "text": "Fifth relevant paragraph...",
    "score": 0.701,
    "document_id": "uuid-456",
    "filename": "textbook_chapter.pdf"
  }
]
```

Important:

- Always return exactly 5 results (or fewer if less than 5 paragraphs exist)
- Results must be sorted by score (highest first)
- Scores should be between 0.0 and 1.0 (cosine similarity, dot product, etc.)

- Only search user's own documents

5 System Architecture Guidelines

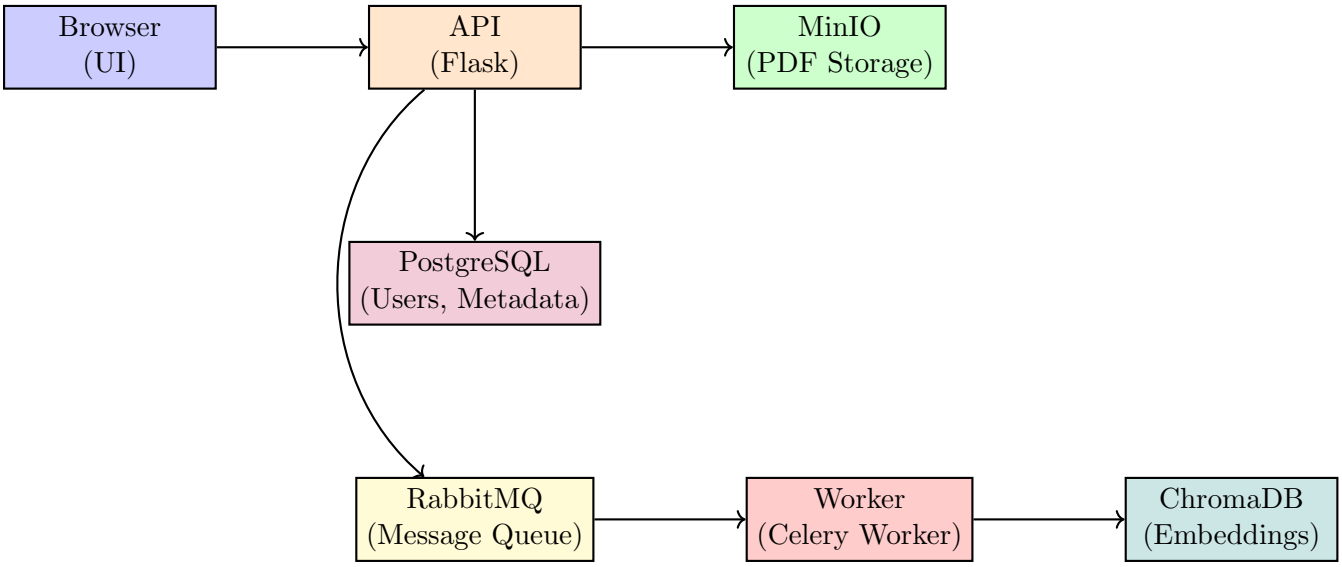
5.1 Required Architecture Patterns

Your system must address these scalability challenges:

1. **Stateless API Tier**
   - API servers should not store files locally
   - Session state in database or Redis (not in-memory)
   - Multiple API instances should work identically
2. **Decoupled Background Processing**
   - PDF upload endpoint returns immediately (202 Accepted)
   - Parsing, chunking, and embedding generation happen asynchronously
   - Use message queue or task queue (RabbitMQ, Celery, Redis Queue)
   - Worker processes can scale independently from API
3. **Persistent Storage Strategy**
   - PDFs: Object storage (MinIO recommended)
   - User data: Relational database (PostgreSQL, MySQL)
   - Vector embeddings: Vector database (ChromaDB, Qdrant, Milvus, pgvector)
   - Session/auth tokens: Database or Redis
4. **Concurrent Request Handling**
   - System must handle multiple users uploading PDFs simultaneously
   - Searches must not block uploads or other searches
   - No race conditions (e.g., two users uploading same filename)
   - **Important:** Consider the case where a single user has two active sessions logged in simultaneously - race conditions on files or database records must be handled properly

5.2 Example Architecture Diagram

Your report must include a detailed architecture diagram. Here's a simplified example:



IMPORTANT: This is just one possible random design example with **no specific reasoning behind it. Your architecture will likely differ significantly.** This is shown only to illustrate what an architecture diagram looks like. Design your own system based on your requirements and justify every choice in your report.

6 Performance Evaluation

6.1 Load Testing Requirements

Your system will be stress tested by TAs on UVic cluster hardware. Tests will include:

1. **Concurrent Uploads**
   - Multiple users uploading PDFs simultaneously
   - PDF sizes: 1MB - 50MB
   - Endpoint must return 202 quickly (target: < 500ms)
2. **Concurrent Searches**
   - Multiple users searching simultaneously
   - Search endpoint latency (target: P95 < 2 seconds)
   - System should not crash or timeout
3. **Mixed Workload**
   - Some users uploading, others searching
   - System should remain responsive
   - No request failures due to resource exhaustion

6.2 What You Must Report

In your video and report, demonstrate:

- **Hardware specifications** you tested on (CPU, RAM, disk)
- **Load testing tool** used (Locust, k6, Apache Bench, etc.)
- **Performance metrics:**
  - Throughput (requests per second)
  - Latency (mean, P50, P95, P99)
  - Maximum concurrent users handled
  - Point of failure (if system crashes, when and why?)
- **Bottleneck analysis:**
  - What was the bottleneck? (CPU, memory, database, vector search?)
  - How did you identify it? (profiling, monitoring)
  - What optimizations did you implement?
  - Before/after performance comparisons

Pro Tip: Start load testing early! Don't wait until the last week. Finding bottlenecks takes time, and fixing them takes even longer.

7 Deliverables

7.1 Docker Compose (Mandatory)

- Single `docker-compose.yml` file
- Must start entire system with: `docker-compose up`
- All services must be containerized (API, worker, databases, MinIO, etc.)
- Must work on **any machine** (no hardcoded paths, no local dependencies)
- Include `.env.example` file with all required environment variables
- Include `README.md` with setup instructions

TAs will test your system by:

1. `git clone <your-repo>`
2. `cp .env.example .env`
3. `docker-compose up`
4. Running automated API tests against `http://localhost:8080`

If this doesn't work, you will lose significant points.

7.2 Technical Report (Mandatory Prerequisite)

IMPORTANT: The report is **mandatory** but **not graded separately**. However, TAs will use your report to evaluate your **Design & Implementation (40%)**.

Without a report, we cannot grade your design decisions, and you will receive 0/40 for Design & Implementation.

Report Requirements:

- Format: PDF only
- Template: Word document template will be provided
- Length: As needed to cover all required content (avoid unnecessary verbosity)
- Must be concise, technical, and well-organized
- Must include:
  - **System architecture diagram** (clear, detailed, professionally drawn)
  - **Design decisions with engineering reasoning** (why you chose each technology, not just what)
  - **Technology stack justification** (backed by research and logic)
  - **Scalability analysis** (how your system scales horizontally)
  - **Bottleneck identification and solutions** (profiling evidence, before/after metrics)
  - **Performance testing methodology** (what tools, what scenarios)
  - **Performance results** (actual numbers: throughput, latency, graphs)
  - **Hardware specifications** (what you tested on)
  - **Load testing graphs and analysis** (visual evidence of performance)
  - **Team collaboration workflow** (how you worked together)
  - **Git workflow explanation** (branching strategy, code review process)
  - **AI usage disclosure** (if applicable - see AI policy section)
  - **Challenges encountered and solutions** (technical depth, problem-solving)

Report Writing Guidelines:

- **Be concise and technical** - Avoid long, verbose, AI-generated content
- **Use diagrams, tables, and graphs** - Not walls of text
- **Include actual numbers and metrics** - Vague descriptions will not receive credit
- **Explain *why* you made decisions** - Not just *what* you did
- **Show engineering depth and critical thinking** - We want to see your reasoning
- **Avoid fluff and padding** - Quality over quantity
- **Cite all sources** - Including AI tools (see AI policy)

What We're Looking For:

- Deep understanding of scalability principles
- Evidence-based decision making
- Professional technical writing
- Critical analysis of your own system
- Honest assessment of limitations and trade-offs

7.3 Presentation Video (15%)

Requirements:

- Length: 10-12 minutes
- Format: MP4, uploaded to YouTube/Google Drive (unlisted or public)
- Both team members must appear and speak
- **No slides required** - present by running your system in real-time
- **Do not speed up video (2x) or rush** - speak clearly and at normal pace
- Must demonstrate:
  1. System architecture overview (2-3 minutes)
     - Show and explain your architecture diagram
     - Walk through your codebase structure briefly
     - Explain key design decisions
  2. Live system demo (3-4 minutes)
     - Sign up / log in
     - Upload PDF in real-time
     - Show processing happening (background worker)
     - Perform search and show results
     - Demonstrate concurrent operations if possible
  3. Performance testing demo (2-3 minutes)
     - Show load testing tool running live
     - Display metrics and graphs
     - Explain bottlenecks you identified
     - Show before/after optimization comparisons
  4. Key technical challenges and solutions (2-3 minutes)
     - Discuss hardest problems you faced
     - Explain how you debugged and solved them
     - Show evidence of problem-solving

Video Grading Criteria:

- Clear audio and screen recording
- Both members contribute equally
- Demonstrates working system live (not just screenshots)
- Shows technical depth and understanding
- Professional presentation (rehearsed, organized, clear speech)
- Evidence of real-time execution (not pre-recorded results)

7.4 GitHub Repository (15%)

IMPORTANT: TAs will carefully review your GitHub repository history to evaluate collaboration, workflow, and contribution patterns. Your commit timeline and messages are evidence of your work process.

Repository Requirements:

- Private repository shared with instructor and TAs
- Professional README with setup instructions and architecture overview
- `.gitignore` file (no PDFs, datasets, or secrets committed)
- Feature branch workflow (no direct commits to `main`)
- Meaningful commit messages that explain what and why
- Both team members must have significant commit history throughout the project
- Use Pull Requests for code reviews before merging
- (Optional but recommended) Use GitHub Issues for task tracking

PROHIBITED:

- **No bulk commits** - Uploading entire project in one or two commits will result in significant point deductions
- **No bulk project uploads** - We expect to see incremental development over time
- **No last-minute commit surges** - Commit history should show consistent work, not all work in final 3 days
- **No unbalanced contributions** - Both team members must contribute substantially

TAs will check commit timestamps, file change patterns, and code authorship. Violations will result in loss of all GitHub & Collaboration points (15%).

Component	Points	Percentage
Design & Implementation	40	40%
- System architecture	12	
- Technology choices	8	
- Scalability patterns	10	
- Code quality & structure	10	
Performance & Load Testing	15	15%
- Load testing implementation	5	
- Performance metrics	5	
- Bottleneck analysis	5	
GitHub & Collaboration	15	15%
- Feature branch workflow	5	
- Commit history & messages	5	
- Code reviews & PRs	5	
Presentation Video	15	15%
- System demo	5	
- Technical depth	5	
- Presentation quality	5	
Checkpoint (March 25)	15	15%
- Docker Compose works	5	
- Basic API functional	5	
- One PDF upload & search	5	
Total	**100**	**100%**
Bonus (TA Discretion)	**up to 20**	**+20%**
- Exceptional optimization		
- Advanced features		
- Outstanding engineering		

Table 1: Grading Breakdown

8 Grading Rubric

8.1 Design & Implementation (40 points)

This is evaluated based on your **technical report** and **code review**. TAs will assess:

- **System Architecture (12 points)**
  - Is the architecture diagram clear and detailed?
  - Does it show all components and data flow?
  - Are scalability patterns correctly implemented?
  - Is the design stateless and horizontally scalable?
- **Technology Choices (8 points)**
  - Are technology choices justified with engineering reasoning?

  - Are technologies appropriate for the problem?
  - Is there evidence of research and understanding?

- **Scalability Patterns (10 points)**
  - Asynchronous processing implemented correctly?
  - Message queue used effectively?
  - Object storage used for stateless design?
  - Concurrent request handling works correctly?

- **Code Quality & Structure (10 points)**
  - Clean, readable, well-organized code
  - Proper error handling
  - Configuration via environment variables
  - No hardcoded credentials or paths
  - Docker Compose setup is clean

8.2 Performance & Load Testing (15 points)

- **Load Testing Implementation (5 points)**
  - Load testing tool properly configured
  - Realistic test scenarios (uploads, searches, mixed)
  - Multiple concurrent users simulated

- **Performance Metrics (5 points)**
  - Clear metrics reported (throughput, latency, etc.)
  - Hardware specifications documented
  - Graphs and visualizations included
  - Before/after comparisons shown

- **Bottleneck Analysis (5 points)**
  - Bottlenecks correctly identified
  - Profiling tools used (cProfile, monitoring, etc.)
  - Root cause analysis demonstrated
  - Optimization attempts documented

8.3 GitHub & Collaboration (15 points)

- **Feature Branch Workflow (5 points)**
  - No direct commits to `main`
  - Feature branches used consistently
  - Branches named meaningfully

- **Commit History & Messages (5 points)**
  - Both members have substantial contributions
  - Commit messages are clear and descriptive
  - Commits are atomic (not huge monolithic commits)
  - Timeline shows consistent work (not last-minute rush)
- **Code Reviews & Pull Requests (5 points)**
  - Pull requests used for merging
  - PRs include descriptions of changes
  - Code review comments present
  - Professional collaboration demonstrated

8.4 Presentation Video (15 points)

- **System Demo (5 points)**
  - Live demo of working system
  - UI and API demonstrated clearly
  - All core features shown
- **Technical Depth (5 points)**
  - Architecture explained clearly
  - Technical challenges discussed
  - Performance results presented
  - Engineering insights demonstrated
- **Presentation Quality (5 points)**
  - Clear audio and visuals
  - Both members contribute equally
  - Well-organized and rehearsed
  - Professional delivery

8.5 Checkpoint: March 25, 2026 (15 points)

Checkpoint Requirements:

1. **Docker Compose Works (5 points)**
   - `docker-compose up` starts all services
   - No manual setup steps required
   - All containers start without errors
2. **Basic API Functional (5 points)**

- `POST /auth/signup` works
- `POST /auth/login` returns token
- `POST /documents` accepts PDF upload
- API responds on port 8080

3. **One PDF Upload & Search (5 points)**
   - Can upload at least one PDF
   - PDF is stored (MinIO or local, doesn't matter yet)
   - `GET /search?q=test` returns results
   - Results don't have to be perfect (even random results OK for checkpoint)

Checkpoint Goal: Demonstrate you have the basic infrastructure in place and are making progress. It's OK if semantic search isn't working perfectly yet, but the core API flow should be operational.

8.6 Bonus Points (Up to 20 points)

TAs may award bonus points for exceptional work beyond core requirements:

- Exceptional performance optimization (e.g., 10x+ improvement over baseline)
- Advanced features beyond requirements (e.g., hybrid search, reranking, query suggestions)
- Outstanding system design and architecture (exceptionally clean and scalable)
- Comprehensive monitoring and observability (Prometheus, Grafana, dashboards)
- Exceptionally high code quality, testing, and documentation
- Creative solutions to complex scalability problems
- Advanced optimizations (caching strategies, database sharding, etc.)

Note on Load Balancing: Implementing a load balancer with multiple API instances is **expected for horizontal scalability**, not bonus. Bonus points are for work that goes significantly beyond baseline expectations.

Note: Bonus points are discretionary and not guaranteed. Focus on meeting core requirements first.

9 Timeline

IMPORTANT - NO EXTENSIONS:

The April 20th deadline is firm and **cannot be extended**. This is the final date for submitting grades to the university. Plan your work accordingly and do not expect any deadline extensions regardless of circumstances.

You have 60 days from release to final submission. Use your time wisely.

Date	Milestone
February 10 (Week 6)	**Project Released**
February 15-20	Team formation, architecture design
February 25	Basic Docker environment and API skeleton
March 5	Core features implementation
March 15	Load testing phase begins
March 25	**Checkpoint Due (15%)**
March 30	Optimization and bottleneck fixing
April 5	Report writing begins
April 10	Final testing and video recording
April 15	Final polish and documentation
April 20	**Final Submission Deadline**
Week of April 21	TAs evaluate and stress test systems

Table 2: Project Timeline (60 Days Total)

Suggested Time Allocation:

- Week 1-2 (Feb 10-23): Architecture design, Docker setup, basic API
- Week 3-4 (Feb 24-Mar 9): Core feature implementation
- Week 4-5 (Mar 10-23): Integration, testing, checkpoint preparation
- Week 5-6 (Mar 24-Apr 6): Load testing, optimization, bottleneck fixes
- Week 7-8 (Apr 7-20): Report writing, video recording, final polish

Critical: Don't leave load testing and optimization until the last week. Finding and fixing bottlenecks takes significant time.

10 GitHub Requirements

10.1 Repository Setup

- Create a **private** repository
- Add instructor and TAs as collaborators
- Repository name: `seng468-semantic-retrieval-<team-names>`

10.2 Required Files

- `README.md` - Setup instructions, architecture overview
- `docker-compose.yml` - Single-command deployment
- `.env.example` - Environment variables template
- `.gitignore` - Exclude PDFs, datasets, secrets, `__pycache__`, etc.

- `requirements.txt` or `package.json` - Dependencies

10.3 Branch Strategy

Required Workflow:

1. Create feature branches from `main`
2. Branch naming: `feature/pdf-upload`, `fix/search-bug`, etc.
3. Work on feature branch
4. Create Pull Request to merge into `main`
5. Other team member reviews and approves
6. Merge to `main`
7. Delete feature branch

NO direct commits to main branch!

10.4 Commit Best Practices

Good commit messages:

```
feat: implement PDF upload endpoint with MinIO storage
fix: resolve race condition in concurrent search queries
refactor: extract embedding logic into separate worker
docs: update README with architecture diagram
perf: optimize vector search with batch processing
```

Bad commit messages:

```
Update
Fixed stuff
WIP
asdfasdf
final version (there should be no "final version" in git!)
```

10.5 What NOT to Commit

- PDFs or test datasets (use `.gitignore`)
- API keys, passwords, secrets (use environment variables)
- Large binary files
- `__pycache__`, `node_modules`, `.DS_Store`
- IDE-specific files (`.vscode`, `.idea`)
- Database files (`*.db`, `*.sqlite`)

11 Report Structure Guidance

Your technical report should be well-organized and cover all required content. Below is the expected structure. A Word document template will be provided to help you organize these sections.

11.1 Required Report Sections

1. **Executive Summary**
   - High-level system overview
   - Key technical decisions made
   - Performance summary and outcomes
2. **System Architecture**
   - Detailed architecture diagram (professionally drawn)
   - Component descriptions and responsibilities
   - Data flow explanation (how data moves through your system)
   - Technology stack justification (why you chose each technology)
3. **Design Decisions & Engineering Reasoning**
   - Why you chose specific technologies
   - Trade-offs you considered (pros/cons of alternatives)
   - Scalability patterns implemented and why
   - Engineering reasoning backed by research and logic
4. **Implementation Details**
   - PDF processing pipeline (parsing, chunking strategy)
   - Embedding generation approach (which model, why)
   - Vector storage and search implementation
   - Authentication and security measures
   - Concurrency handling (race conditions, simultaneous sessions)
5. **Performance Testing & Bottleneck Analysis**
   - Hardware specifications you tested on
   - Load testing methodology (tools, scenarios)
   - Performance metrics with actual numbers (throughput, latency, graphs)
   - Bottleneck identification (profiling evidence, monitoring data)
   - Optimization attempts and before/after results
   - System limitations and capacity bounds
6. **Technical Challenges & Problem Solving**
   - Specific technical challenges encountered

- How you debugged and solved them (show problem-solving process)
- What didn't work and why
- Key lessons learned

7. **Team Collaboration & Workflow**
   - Task division between team members
   - Communication and coordination approach
   - Git workflow explanation (branching, PRs, code reviews)
   - How you resolved conflicts or disagreements
8. **AI Usage Disclosure** (if applicable)
   - Which AI tools used (specific names)
   - What you used them for (code, debugging, research, writing)
   - Approximate percentage of AI vs. your own work
   - Examples of AI-generated content you reviewed/modified
   - Evidence of your understanding

Important: Focus on quality and completeness of content, not page count. Include all required sections with sufficient detail to demonstrate your engineering depth and decision-making process.

12 AI Usage Policy

12.1 AI Tools Are Allowed With Disclosure

You are **permitted to use AI tools** such as:

- ChatGPT, Claude, Gemini, other LLMs
- GitHub Copilot, Cursor, other code assistants
- Code completion and suggestion tools

12.2 Core Principle: You Control AI, Not Vice Versa

Use AI as a Tool, Not a Replacement for Your Engineering Judgment

- AI should **facilitate your work** - help you learn, search, debug, write faster
- AI should **not think for you** - you must understand every line of code and every design decision
- **You are engineers** under evaluation, not AI output reviewers
- Use AI **supervised under your control**, not blindly following suggestions
- This project is for **your assessment and learning** - it should fundamentally be **your work**

12.3 Mandatory Disclosure

You MUST clearly cite all AI contributions in your report. Include:

1. Which AI tools you used (specific names and versions)
2. What you used them for specifically:
   - Code generation (which parts?)
   - Debugging (which issues?)
   - Research and learning (which topics?)
   - Report writing (which sections?)
   - Architecture suggestions (which decisions?)
3. Approximate percentage of work done by AI vs. your own work
4. Examples of AI-generated code that you reviewed and modified
5. Evidence that you **understand** all AI-generated code
6. How you verified AI suggestions were correct

Failure to disclose AI usage is academic misconduct and will result in 0 for the project.

12.4 Understanding Is Required

- You must **fully understand all code** in your project, regardless of source
- TAs **may ask you to explain any part** of your code during evaluation
- If you cannot explain code in your submission, you will lose substantial points
- AI-generated code must be **reviewed, tested, understood, and often modified** by you
- Blindly copying AI output without understanding shows lack of engineering competence

12.5 Acceptable vs. Unacceptable AI Use

Acceptable Use:

- Using AI to learn about vector databases you haven't seen before
- Asking AI to explain error messages and suggest debugging approaches
- Using AI to generate boilerplate code that you then customize
- Using AI to review your architecture and suggest improvements
- Using AI to help format and structure your report
- Using AI as a "rubber duck" to discuss design trade-offs

Unacceptable Use:

- Asking AI to generate entire project and submitting with minimal changes
- Copying AI-generated code without understanding how it works
- Letting AI make all architectural decisions without your engineering judgment
- Submitting AI-written report sections verbatim without editing
- Using AI to write your video script word-for-word
- Not disclosing AI usage at all
- Claiming AI-generated work as entirely your own

Best Practice: Think of AI as an experienced colleague you can consult, not as someone who does your work for you. The best projects will show clear evidence of **your own engineering judgment**, **critical thinking**, and **problem-solving** - not just AI output.

Use AI to learn faster and work more efficiently, but make sure the fundamental ideas, decisions, and implementations are yours.

13 Tips for Success

13.1 Start Early

- Don't wait until the checkpoint to start
- Docker setup alone can take hours
- Integrating multiple services is complex
- Load testing and optimization take the most time

13.2 Incremental Development

Build in this order:

1. Basic API with authentication (no PDFs yet)
2. Simple PDF upload (save locally first, MinIO later)
3. PDF parsing and chunking (start with simple logic)
4. Embedding generation (test with small PDFs)
5. Vector storage and search (start with simple vector DB)
6. Asynchronous processing (add queue last)
7. Load testing and optimization

13.3 Test Locally First

- Test each component separately before integration
- Use small test PDFs (1-2 pages)
- Test Docker Compose early (don't leave it to the end)
- Test on a fresh machine (ask a friend to run your Docker Compose)

13.4 Communication

- Meet with your partner regularly (at least twice a week)
- Use GitHub Issues or Trello for task tracking
- Document decisions in commit messages or README
- Ask TAs or instructor if you're stuck (don't waste days debugging)

13.5 Common Pitfalls to Avoid

- **Hardcoding paths:** Use environment variables
- **Ignoring Docker:** Test Docker Compose from day 1
- **Skipping load testing:** Start load testing by Week 3
- **Oversized commits:** Commit frequently with clear messages
- **No documentation:** Document as you go, not at the end
- **Last-minute report:** Start report outline early

14 Frequently Asked Questions

Q: Do we need a UI?
A: No, UI is not graded. A simple UI is recommended for the demo video, but TAs will test via API only.

Q: Can we use [technology X] instead of MinIO/RabbitMQ/etc.?
A: Yes! Technology choices are yours. Just justify them in your report.

Q: What if our vector database doesn't scale well?
A: That's OK. Document the bottleneck in your report, explain what you tried, and discuss what you would do differently with more time/resources.

Q: How many PDFs should our system handle?
A: There's no fixed number. Test with what your hardware can handle. Document your system's limits and explain the bottlenecks.

Q: Can we use cloud services (AWS, GCP, Azure)?
A: No. Everything must run via Docker Compose locally. This ensures fair testing for all teams.

Q: What if we miss the checkpoint?
A: You lose 15% of your grade. No extensions except for documented emergencies.

Q: Can we use pre-trained embedding models?
A: Yes! Use sentence-transformers, OpenAI, or any embedding model. Just document your choice.

Q: How do we handle large PDFs (100+ pages)?
A: Implement chunking (split into paragraphs or fixed-size chunks). Document your chunking strategy.

Q: What if Docker Compose doesn't work on the TA's machine?
A: You will lose significant points. Test on multiple machines before submitting.

15 Submission

15.1 What to Submit (April 20, 2026)

Submit via Canvas:

1. **GitHub repository URL** (ensure TAs have access)
2. **Technical report PDF** (15-20 pages)
3. **Video presentation link** (YouTube/Google Drive)
4. **.env.example file** (as a separate file for TA convenience)

15.2 GitHub Repository Must Contain

- All source code
- `docker-compose.yml`
- `README.md` with setup instructions
- `.env.example`
- `.gitignore`
- `requirements.txt` or dependency file

15.3 Late Policy

- **Checkpoint (March 25):** No late submissions accepted
- **Final submission (April 20):** -10% per day late (max 3 days)
- After April 23: Project receives 0

16 Academic Integrity

- You may discuss high-level ideas with other teams
- You may NOT share code with other teams
- You MUST disclose AI usage in your report
- Both team members must contribute substantially
- TAs will review commit history for evidence of collaboration

Violations will result in a grade of 0.

Good luck! This is your chance to showcase your scalability skills.

Build something you're proud to put on your resume.